



UNIVERSITÀ DEGLI STUDI DI UDINE
DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE
CORSO DI LAUREA IN INFORMATICA

Gli algoritmi di ordinamento

Stima e analisi dei tempi medi di esecuzione di
QuickSort, QuickSort 3-Way, CountingSort e IntroSort

Relazione per il corso di **Algoritmi e Strutture Dati**

Studente:

Lisa Andreon

Matricola: 167335

167335@spes.uniud.it

Studente:

Francesca Sarri

Matricola: 169172

169172@spes.uniud.it

Referente del progetto: Lisa Andreon

Anno Accademico 2024/2025

Indice

1	Introduzione	2
1.1	Specifiche tecniche del progetto	2
1.2	Premesse	3
2	Organizzazione del codice	3
2.1	Gestione del calcolo dei tempi medi di esecuzione	4
2.2	Note importanti	5
3	<i>QuickSort</i>	5
3.1	Complessità	5
3.2	Grafici in funzione di n	6
3.3	Grafici in funzione di m	7
4	<i>CountingSort</i>	8
4.1	Complessità	8
4.2	Grafici in funzione di n	9
4.3	Grafici in funzione di m	10
5	<i>QuickSort 3-Way</i>	11
5.1	Complessità	11
5.2	Grafici in funzione di n	12
5.3	Grafici in funzione di m	13
6	<i>IntroSort</i>	14
6.1	Pseudo-codice	14
6.2	Complessità	14
6.3	Grafici in funzione di n	16
6.4	Grafici in funzione di m	17
7	Confronto tra algoritmi	19
7.1	<i>QuickSort</i> VS <i>QuickSort 3-Way</i>	19
7.2	<i>IntroSort</i> VS <i>CountingSort</i>	20
7.3	<i>QuickSort</i> VS <i>CountingSort</i>	21
7.4	Tutti	22
7.4.1	Grafici in funzione di n	22
7.4.2	Grafici in funzione di m	24

1 Introduzione

Il seguente progetto ha lo scopo di concretizzare il funzionamento e la complessità di alcuni algoritmi di ordinamento, in base allo studio dei loro tempi medi di esecuzione e all'analisi dei risultati ottenuti. Sono stati presi in esame i seguenti algoritmi: *QuickSort*, *CountingSort*, *QuickSort 3-Way* e, come algoritmo a scelta, *IntroSort*.

Quest'ultimo, in particolare, è stato scelto per i seguenti motivi:

- *IntroSort* è un algoritmo ibrido che combina *QuickSort*, *InsertionSort* e *HeapSort*, quindi risulta interessante lo studio della sua complessità;
- esso garantisce buone prestazioni, evitando il caso peggiore di *QuickSort*;
- solitamente, in caso di *array* o partizioni di piccole dimensioni si rivela più veloce del *QuickSort* o dell'*HeapSort* a causa del suo basso *overhead*;
- è l'algoritmo di ordinamento predefinito in molte librerie standard di numerosi linguaggi di programmazione, come ad esempio *Python*.

1.1 Specifiche tecniche del progetto

Per prima cosa il progetto è stato strutturato con l'**implementazione degli algoritmi di ordinamento**, in funzione della dimensione dell'*array* e del range di interi su cui gli elementi di quest'ultimo possono variare.

Quindi, si hanno i seguenti **parametri**:

- n , ovvero la lunghezza dell'*array*, che deve essere compresa in un intervallo di valori tra 100 e 100.000.
- m , la dimensione dell'intervallo di interi, che deve essere compresa tra 10 e 1.000.000.

In seguito, è stato sviluppato un programma in grado di calcolare, in primis, i **tempi di esecuzione medi** degli algoritmi, con la possibilità di scegliere quale parametro mantenere costante (*fixed*), con la possibilità di generare due tipologie di grafici:

- una in funzione della lunghezza dell'*array* (n), in cui il parametro della dimensione dell'intervallo di interi (m) è stato fissato a 100.000;
- l'altra in funzione della dimensione dell'intervallo di interi (m), con il parametro della lunghezza dell'*array* (n) fissato a 10.000.

In secondo luogo, è stato creato un ulteriore programma per la **creazione dei grafici**, prelevando da una lista, i campioni necessari per la visualizzazione della complessità degli algoritmi presi in esame; nella sezione 2 verrà spiegato più nel dettaglio quanto scritto.

1.2 Premesse

Per generare i grafici, si è scelto di utilizzare il dispositivo di Francesca, al posto di quello di Lisa, poiché l'esecuzione dei vari programmi si è dimostrata più stabile e con meno "sbavature".

Questo comportamento anomalo potrebbe essere stato causato dall'interferenza dei processi in *background*, disturbando i tempi di esecuzione dei dati, e dalle differenti specifiche tecniche dei dispositivi: l'*hardware* di Francesca ha predisposto un ambiente più stabile per il *benchmarking*, permettendo di ottenere risultati più coerenti.

2 Organizzazione del codice

Il progetto è stato suddiviso in diversi file, in modo tale da avere un'organizzazione chiara e semplificativa in caso di manutenzione, ma anche per separare le varie responsabilità. Nello specifico:

- `countingsort.py`, `introsort.py`, `quicksort.py` e `quicksort_3way.py`: corrispondono agli algoritmi di ordinamento del nostro progetto;
- `main_grafici.py`: contiene la logica principale e il punto di ingresso del programma;
- `plotter_grafici.py`: gestisce la creazione dei grafici;
- `calcolo_benchmark.py`: contiene le funzioni per misurare il tempo di esecuzione.

A partire dal `main()`, è possibile usufruire della funzione `compare_plot_algoritmi(...)`, che permette di creare grafici, sia singoli sia comparativi, esplicitando in un vettore di stringhe i nomi degli algoritmi. Questi ultimi referenziano ai corrispondenti codici eseguibili degli algoritmi mediante l'implementazione di un dizionario, denominato `ALGORITMI_CONFIG`, nel quale ciascuna chiave corrisponde ad un oggetto di tipo `AlgConfig`.

La creazione di questa classe ha l'obiettivo di evitare la duplicazione del codice per ogni algoritmo e di rendere dinamica la generazione di grafici, in grado di confrontare più algoritmi. Tale approccio permette, inoltre, di gestire in modo flessibile la diversificazione dei grafici in base al parametro n o m .

È stata implementata, inoltre, un'ulteriore classe, denominata `BenchmarkConfig`, con lo scopo di modificare facilmente i parametri principali, quali il numero di ripetizioni e il numero di campioni. Sempre al suo interno, è stato dichiarato il *flag* `AVVIO_BENCHMARK`, il quale può avere uno dei seguenti stati:

- `TRUE`: misura i tempi medi dell'algoritmo scelto e salva i risultati ottenuti in un *file CSV*;
- `FALSE`: viene effettuato direttamente il caricamento, a partire dal *file CSV* corrispondente all'algoritmo in input, prelevando le coppie dei campioni e inserendole in una lista di tuple.

Infine è possibile creare grafici sia in scala lineare sia doppiamente logaritmica, la cui analisi verrà discussa nelle sottosezioni dei vari algoritmi.

2.1 Gestione del calcolo dei tempi medi di esecuzione

La funzione `esegui_benchmark(...)`, presente nel file `calcolo_benchmark.py`, permette di misurare il tempo di esecuzione medio dell'algoritmo preso in esame, il quale è opportunamente passato come parametro alla suddetta funzione, come oggetto `AlgConfig`.

Il tempo di esecuzione varia in base al parametro scelto come costante, determinando se quest'ultimo è in funzione di n o m ; in ogni caso, vengono creati 250 campioni, identificati dalla coppia (n o m , tempo medio): essi costituiscono i punti presenti nei grafici finali.

In questo progetto, si è deciso di utilizzare una progressione geometrica per generare i valori da testare. Questo per garantire una copertura ampia, in particolar modo concentrando più punti nei valori piccoli, dove spesso si osservano i cambiamenti più rilevanti, e meno nei valori grandi, dove l'aumento del tempo è più prevedibile. In generale, consente di distribuire i punti in maniera più leggibile per un'analisi precisa.

Nel nostro caso si hanno 250 campioni, generati dalla seguente formula:

$$n_i = A \cdot B^i$$

oppure $m_i = A \cdot B^i$

dove:

- i : è un indice compreso fra 0 e 249;
- A : è il primo valore di n o m , a seconda del parametro variabile (`val_min`);
- B : è la ragione geometrica.

L'obiettivo è quello di calcolare B (nel caso in cui n è la variabile):

$$\begin{aligned} A \cdot B^{249} &= 100000 \quad (\text{val_max di } n) \\ 100 \cdot B^{249} &= 100000 \end{aligned}$$

Troviamo quindi B :

$$B = \left(\frac{100000}{100} \right)^{\frac{1}{249}}$$

Questo si traduce nel caso generico:

$$\begin{aligned} A &= \text{val_min} \\ B &= \left(\frac{\text{val_max}}{\text{val_min}} \right)^{\frac{1}{\text{num_campioni}-1}} \end{aligned}$$

ovvero, con `num_campioni` punti, si esprime come:

$$n_i = \text{min_val} \cdot \left(\frac{\text{max_val}}{\text{min_val}} \right)^{\frac{i}{\text{num_campioni}-1}}$$

Equivalente, nel caso di m_i .

2.2 Note importanti

- Per utilizzare gli stessi dati (i numeri generati nei vettori) ogni volta che si creano i grafici, è stato specificato il `seed(2)`, ovvero un numero (seme), in grado di generare un ulteriore numero pseudo-randomico, con lo scopo di testare gli algoritmi con gli stessi dati casuali, ogni volta che viene lanciata la funzione generatrice di grafici. I risultati permangono sia tra le diverse chiamate al singolo lancio del programma, sia se quelle stesse chiamate sono state lanciate in un secondo momento.
- Essendo *QuickSort* un algoritmo ricorsivo e meno efficiente di *QuickSort 3-Way*, nel caso peggiore vengono fatte $O(n)$ chiamate ricorsive, pertanto è stato necessario aumentare il limite di ricorsione, dato che *Python* ha un limite predefinito di circa 1000 ricorsioni. Questo perché, nel caso di grafico in funzione di m , si parte da un range di numeri relativamente basso che porta a dei duplicati. Quindi, per evitare di aumentare il *range* di m , abbiamo optato per impostare un limite di 100000 ricorsioni.

Si noti che *QuickSort 3-Way* non presenta la stessa problematica, in quanto non necessita di ricorsioni aggiuntive per la gestione dei duplicati.

3 *QuickSort*

QuickSort è un algoritmo di ordinamento che segue il paradigma "*Divide et Impera*".

Dato un vettore di interi, seleziona un elemento, denominato *pivot*: questa operazione è fondamentale, perché consente a *QuickSort* di spostare gli elementi minori o uguali al *pivot* alla sua sinistra (di conseguenza, quelli maggiori si troveranno alla sua destra).

A questo punto, l'algoritmo viene applicato ricorsivamente ai due sotto-*array* che si sono formati e termina quando il vettore è ordinato correttamente.

QuickSort non è considerabile un algoritmo stabile, infatti, durante la fase di partizionamento, gli elementi con lo stesso valore potrebbero essere scambiati e addirittura superarsi a vicenda.

D'altro canto, è un algoritmo *in-place*, poiché l'ordinamento avviene direttamente sull'*array* in input, senza la necessità di creare una copia ausiliaria.

3.1 Complessità

La complessità temporale di *QuickSort* varia notevolmente a seconda di come viene scelto il pivot a ogni passaggio:

- nel **caso peggiore**, ovvero quando il vettore è ordinato al contrario o è già ordinato, si ha complessità $\Theta(n^2)$;
- nel **caso migliore**, così come in quello **medio**, si ottiene $\Theta(n \log n)$.

3.2 Grafici in funzione di n

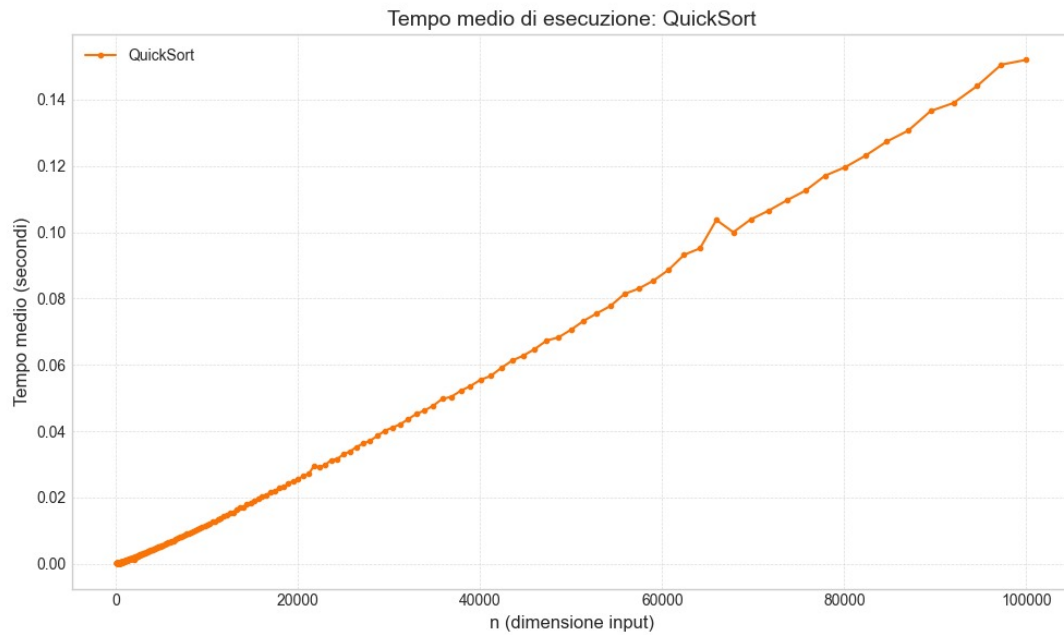


Figura 1: Scala lineare

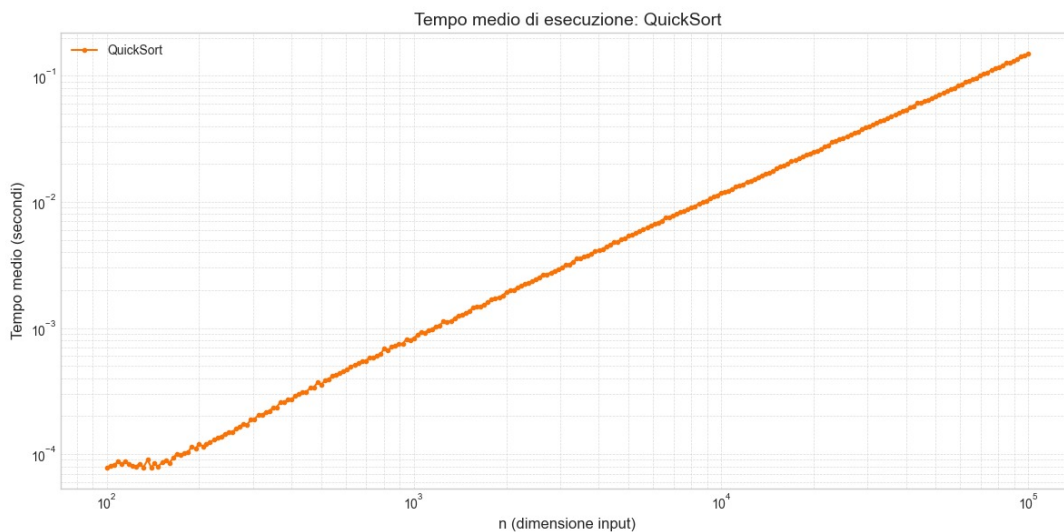


Figura 2: Scala doppiamente logaritmica

- Secondo la complessità teorica menzionata precedentemente, ci si aspetta una complessità nel caso medio di $\Theta(n \log n)$. Quindi, il tempo di esecuzione dovrebbe crescere in modo leggermente superiore rispetto a una funzione lineare.
- Il risultato del grafico è simile a quanto supposto, sebbene la pendenza potrebbe sembrare costante e non concava sul basso; questa discrepanza è probabilmente dovuta dal range di valori di n e dalla scala del grafico, che rendono difficile distinguere visivamente una crescita $n \log n$ da una strettamente lineare, poiché il logaritmo cresce molto lentamente rispetto a n . Però questa differenza impercettibile nel grafico

lineare è visibile chiaramente nel grafico doppiamente logaritmico, dove è possibile notare una forma della curva leggermente concava verso il basso.

3.3 Grafici in funzione di m

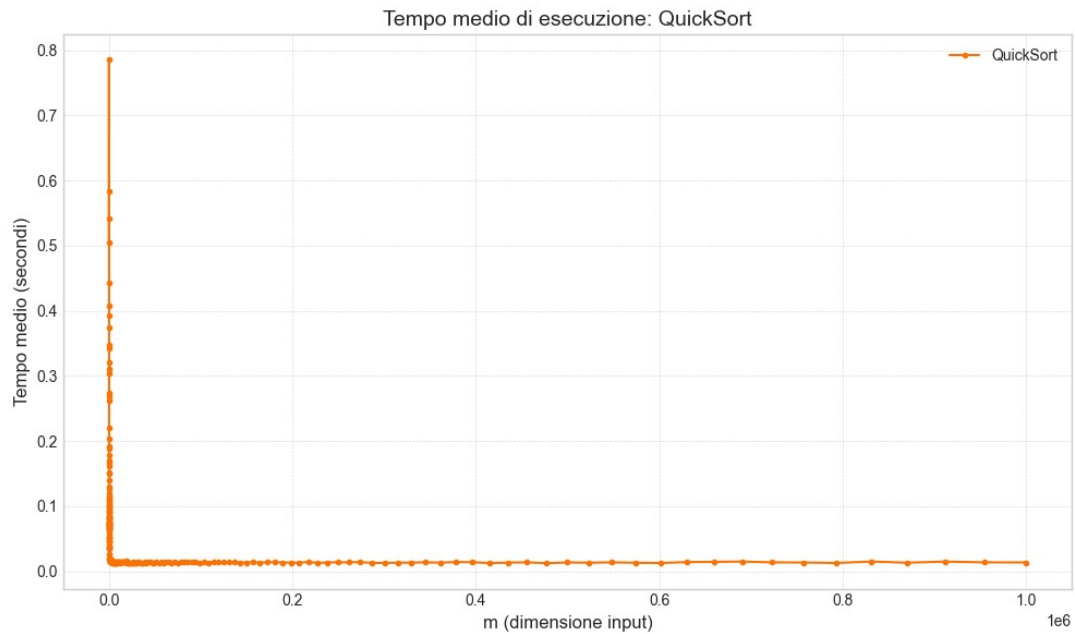


Figura 3: Scala lineare

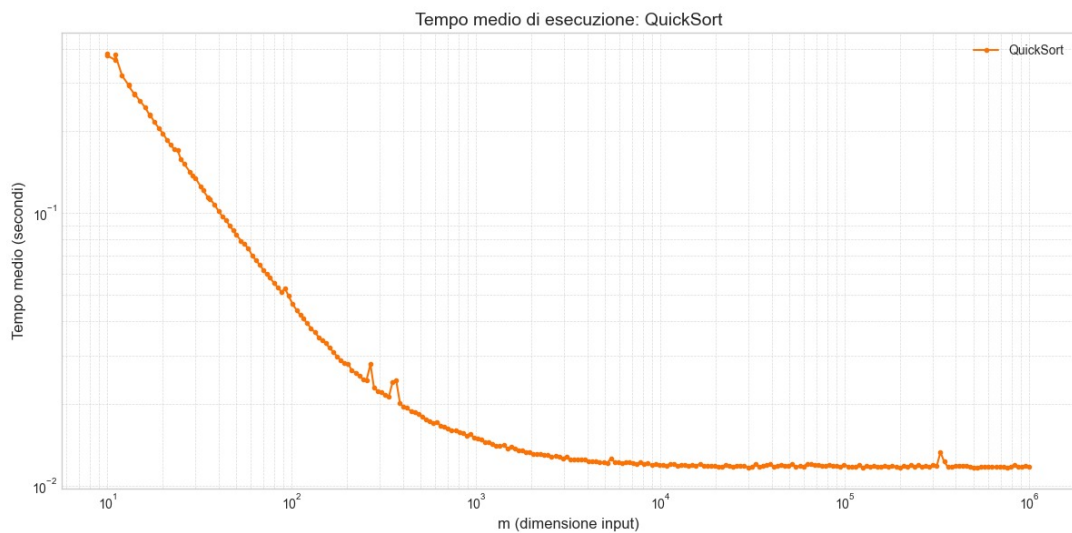


Figura 4: Scala doppiamente logaritmica

- Secondo quanto studiato in precedenza, la complessità teorica temporale di *QuickSort* (ovvero $O(n \log n)$) è asintoticamente indipendente da m , ovvero il range dei valori presenti nell'*array*, poiché dipende esclusivamente dal numero di elementi da ordinare, n . Pertanto, ci si aspetterebbe che il tempo di esecuzione rimanga pressoché costante al variare di m .

- Il grafico mostra una chiara indipendenza dal valore di m , essendo che la linea rimane costante per la maggior parte del tempo. Tuttavia è possibile notare una chiara anomalia quando il valore di m è piccolo: il tempo di esecuzione raggiunge un picco massimo di circa $0,8s$. Questo è probabilmente dovuto al fatto che, il range partendo da un valore di 10 (relativamente basso in confronto ad n), in un vettore di 10.000 elementi, questi valori si ripeteranno molto frequentemente. *QuickSort* non gestisce in modo ottimale gli elementi duplicati, e in questi casi può degenerare in termini di risultati, avvicinandosi al caso peggiore.
- Questa degenerazione è possibile notarla chiaramente nel grafico doppiamente logaritmico, dove la linea parte da un picco molto alto per poi decrescere lentamente sino a stabilizzarsi in una linea quasi orizzontale.

4 *CountingSort*

CountingSort è un algoritmo di ordinamento non basato sul confronto, adatto per organizzare una collezione di oggetti tramite chiavi, cioè piccoli numeri interi; il suo funzionamento si concentra sull'idea di contare quante volte ogni elemento compare nell'*array* di input.

CountingSort è stabile, ciò significa che elementi aventi lo stesso valore appariranno nell'*array* di output con lo stesso ordine di quello in input.

Tuttavia, non è *in-place*: per funzionare infatti, usa un *array* di conteggio di dimensione $k + 1$ e uno di output di dimensione n per memorizzare il risultato ordinato.

4.1 Complessità

Sapendo che *CountingSort* dipende dal numero di elementi da ordinare (n) e dall'intervallo dei valori di input (k), si afferma che, in qualsiasi caso, la sua complessità equivale a $\Theta(n + k)$ in termini di tempo, poiché l'algoritmo esegue sempre la stessa sequenza di operazioni, indipendentemente dalla distribuzione iniziale dei dati.

4.2 Grafici in funzione di n

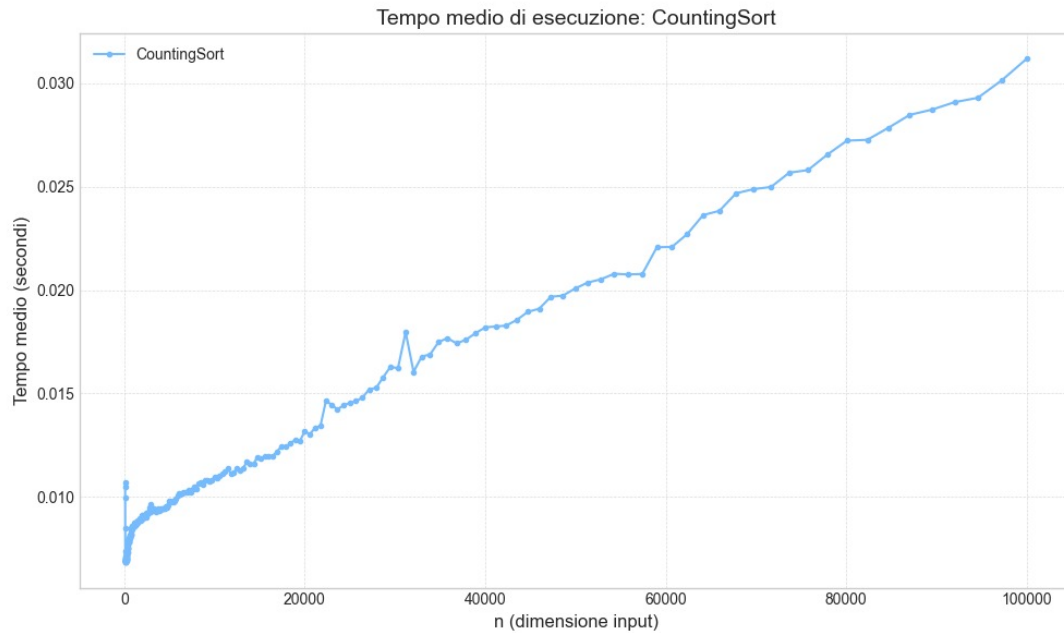


Figura 5: Scala lineare

- Come detto precedentemente, *CountingSort* cresce in modo direttamente proporzionale al numero di elementi n , con m fissato; di conseguenza, si presume che l'andamento sia una linea retta e che la sua pendenza sia positiva e costante.
- La complessità di *CountingSort* è $\Theta(n+m)$ e, poiché m è una costante fissa, il tempo di esecuzione dipende linearmente da n , cioè $\Theta(n + \text{costante})$, che è una retta con una pendenza positiva.



Figura 6: Scala doppiamente logaritmica

- Ci aspettiamo che per $n \ll m$, la costante domini, poiché il tempo di esecuzione è quasi costante e quindi il grafico dovrebbe apparire come una linea orizzontale. Invece, per $n \gg m$, n domina, tant'è che l'andamento è quasi lineare rispetto ad n , diventando una retta con pendenza 1. Infine, nel caso di $n \approx m$, la curva potrebbe non assumere alcuna delle precedenti forme.
- Il grafico ottenuto in scala doppiamente logaritmica di *Counting Sort* in funzione di n rispecchia la sua complessità $\Theta(n + m)$: la curva mostra chiaramente un regime costante quando n è piccolo (dominato da m) e un regime lineare (pendenza 1) quando n diventa grande (dominato da n).
- Da segnalare che nel grafico si è manifestato un picco improvviso, dovuto probabilmente ad un'anomalia durante il processo.

4.3 Grafici in funzione di m

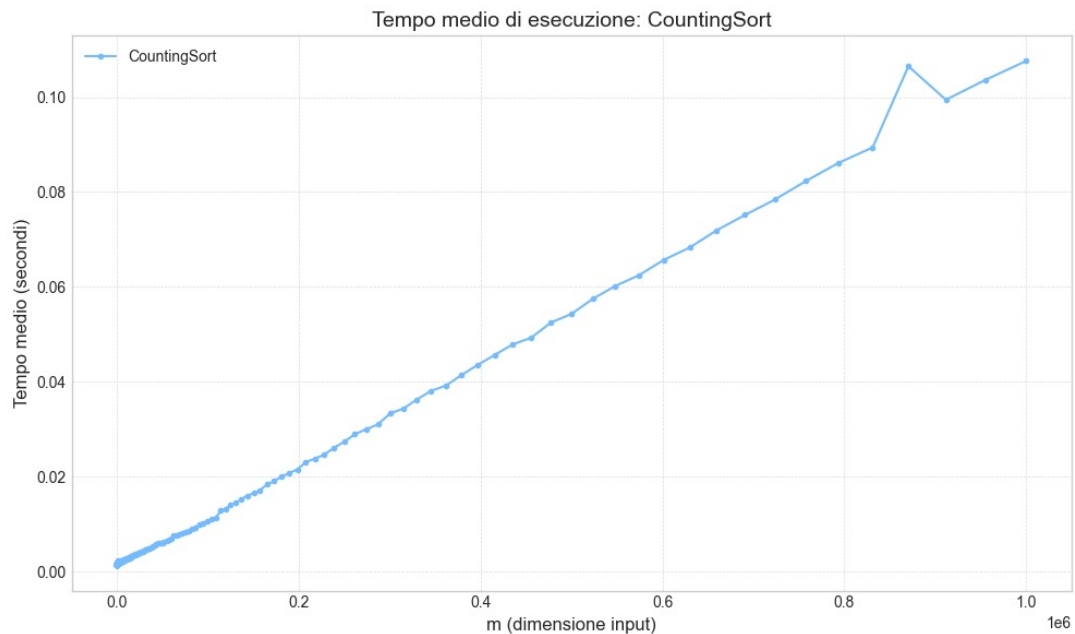


Figura 7: Scala lineare

- Come visto in precedenza nello studio in funzione di n , anche qui ci si aspetta che il grafico sia una linea retta, ed effettivamente è proprio il risultato ottenuto: con n fisso, il tempo di esecuzione cresce linearmente con la dimensione del range m , risultando in una linea retta con pendenza positiva.

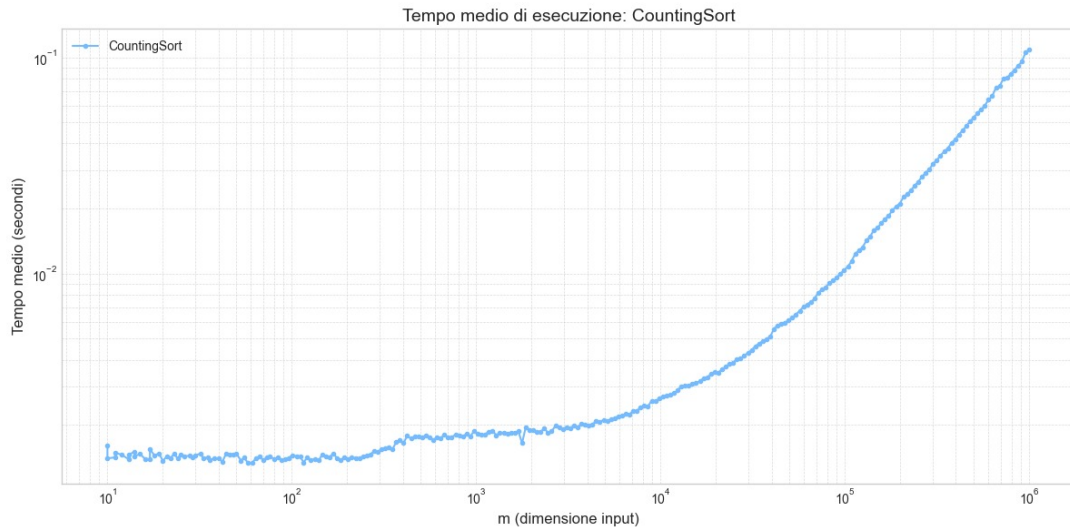


Figura 8: Scala doppiamente logaritmica

- Ci si aspetta che la curva mostri un andamento quasi orizzontale per valori di m piccoli, dove il tempo di esecuzione è dominato dal numero di elementi n ; successivamente, al crescere di m , si prevede che la curva diventi una retta avente pendenza 1. Infine, potrebbe manifestarsi una fase di transizione, in cui la pendenza della curva cambia gradualmente da 0 a 1, indicando che il tempo di esecuzione sta passando dall'essere dominato da n all'essere dominato da m .
- La curva generata mostra chiaramente queste caratteristiche, senza particolari anomalie.

5 *QuickSort 3-Way*

QuickSort 3-Way è una variante specializzata del classico *QuickSort*, che partiziona l'*array* in tre sezioni, in base alla scelta del *pivot*:

- a sinistra, sono sistemati gli elementi minori del *pivot*;
- al centro, quelli uguali;
- a destra, quelli maggiori.

Il suo scopo principale è gestire in modo più efficiente i vettori che contengono molti elementi duplicati e migliorare il costo nel caso peggiore.

QuickSort 3-Way non è stabile: durante il processo di partizionamento, l'ordine di elementi (diversi dal *pivot*) con lo stesso valore può essere alterato.

Ciononostante, è un algoritmo *in-place*, proprio come la versione classica, poiché l'ordinamento avviene modificando direttamente l'*array* di input.

5.1 Complessità

- Nel **caso peggiore**, quando il vettore è già ordinato e il *pivot* è sbilanciato, si ottiene una complessità temporale di *QuickSort 3-Way* pari a $\Theta(n^2)$;

- nel **caso migliore e medio**, invece, $O(n \log n)$.

5.2 Grafici in funzione di n



Figura 9: Scala lineare



Figura 10: Scala doppiamente logaritmica

- Come detto in precedenza, il *QuickSort 3-Way* presenta nel caso medio una complessità teorica di $\Theta(n \log n)$, proprio come nel caso di *QuickSort*. L'ottimizzazione riguarda principalmente le prestazioni in presenza di elementi duplicati, ma non cambia l'ordine di grandezza asintotico. Pertanto, il grafico risulta molto simile a quello analizzato precedentemente nel caso di *QuickSort*.

- In scala doppiamente logaritmica, è possibile notare che la curva non presenta anomalie significative, pertanto il grafico si avvicina ad un andamento quasi lineare e leggermente più efficiente di *QuickSort*.

5.3 Grafici in funzione di m

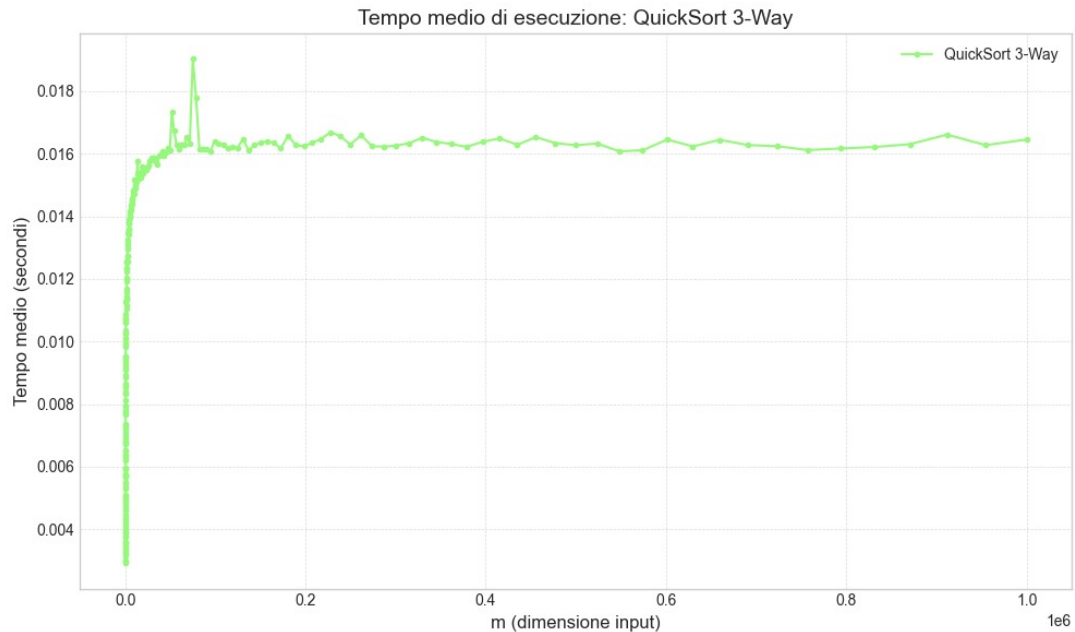


Figura 11: Scala lineare

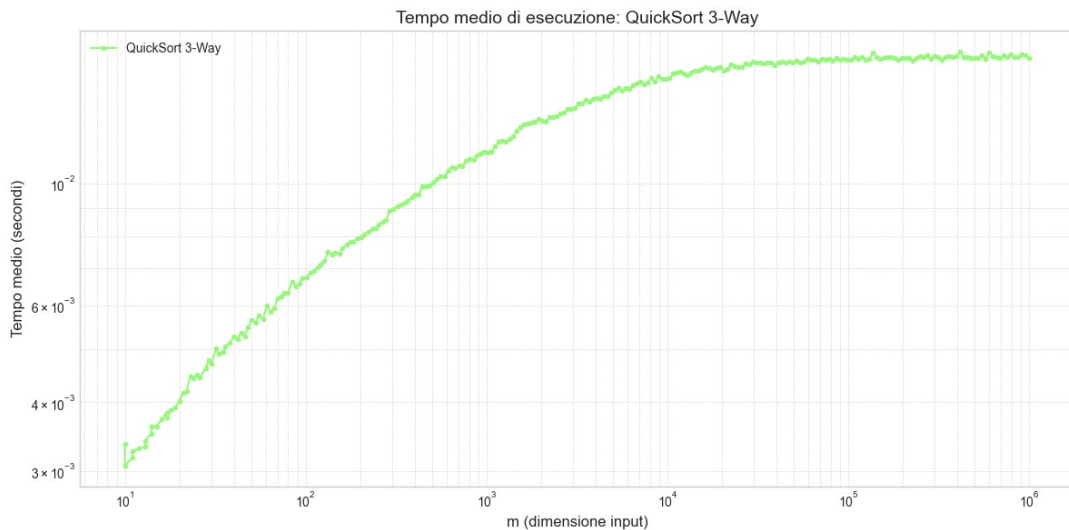


Figura 12: Scala doppiamente logaritmica

- Chiaramente, come nel caso di *QuickSort*, è possibile notare che il tempo di esecuzione risulta asintoticamente indipendente da m , rimanendo costante durante l'intero grafico ($m \approx 0.016$). Tuttavia, all'inizio del grafico è presente un picco iniziale

quando m è piccolo, dovuto probabilmente alla presenza di molti valori duplicati, ma raggiunge il suo valore di stabilità molto rapidamente.

- È possibile confermare quanto detto in precedenza, analizzando il grafico in scala doppiamente logaritmica: vi è un leggero *overhead* per m molto piccolo, ma si stabilizza molto velocemente e gradualmente.

6 *IntroSort*

Come già anticipato nell'introduzione (sezione 1), *IntroSort* è implementato sulla base di un ibrido, che sfrutta la velocità di *QuickSort* nella maggior parte dei casi (tenendo traccia della profondità della ricorsione), evita i suoi scenari peggiori con *HeapSort* e ottimizza l'ordinamento di piccole porzioni di dati con l'*InsertionSort*.

IntroSort non è un algoritmo stabile, poiché sia *QuickSort* sia *HeapSort* non lo sono. Malgrado questo, è un algoritmo *in-place*, infatti, l'ordinamento viene eseguito direttamente sul vettore in input, senza richiederne uno ausiliario di dimensioni proporzionali all'input.

6.1 Pseudo-codice

Algorithm 1 Introsort

```

1: procedure INTROSORT( $A, n$ )                                ▷ array, numero di elementi
2:    $max\_depth \leftarrow 2 \cdot \lfloor \log_2 n \rfloor$               ▷ calcolo profondità massima
3:   INTROSORT_REC( $A, 1, n, max\_depth$ )
4: end procedure
5: procedure INTROSORT_REC( $A, start, end, depth\_limit$ )
6:   if  $(end - start + 1) \leq 16$  then
7:     INSERTIONSORT( $A, start, end$ )                          ▷ per piccoli array
8:   else if  $depth\_limit = 0$  then
9:     HEAPSORT( $A, start, end$ )                                ▷ profondità massima raggiunta
10:  else
11:     $pivot \leftarrow$  PARTITION( $A, start, end$ )
12:    INTROSORT_REC( $A, start, pivot - 1, depth\_limit - 1$ )
13:    INTROSORT_REC( $A, pivot + 1, end, depth\_limit - 1$ )
14:  end if
15: end procedure

```

6.2 Complessità

Per analizzare la complessità di *IntroSort*, è necessario determinare i costi delle singole operazioni, partendo dalla procedura ricorsiva INTROSORT_REC(...), per poi concludere con quella principale INTROSORT(...). Per prima cosa, conviene considerare il caso peggiore.

Di seguito, è sviluppata l'equazione di ricorrenza della procedura `INTROSORT_REC(...)`:

$$T(n, d) = \begin{cases} \Theta(1) & \text{se } n \leq 16 \\ \Theta(n \log n) & \text{se } n > 16 \text{ e } d = 0 \\ T(m, d-1) + T(n-m-1, d-1) + \Theta(n) & \text{se } n > 16 \text{ e } d > 0 \end{cases}$$

Analizzando il terzo caso, ovvero il caso peggiore di *QuickSort*, è necessario considerare che la funzione `PARTITION(...)` produrrà costantemente partizioni massimamente sbilanciate, ovvero restituirà un pivot tale che una partizione abbia 0 elementi e un'altra con $n-1$.

Prendendo il caso di $m = 0$, la terza riga dell'equazione diventa:

$$\begin{aligned} T(n, d) &= T(m, d-1) + T(n-m-1, d-1) + \Theta(n) \\ &= T(n-1, d-1) + \Theta(n) \quad (\text{semplificazione per il primo caso base}) \end{aligned}$$

Successivamente, risolviamo quest'ultima equazione mediante gli alberi ricorsivi, semplificando il termine $\Theta(n \log n) = b \cdot n$:

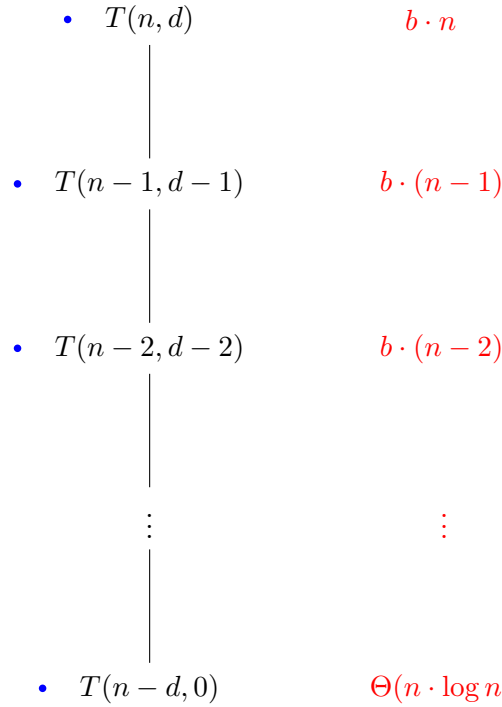


Figura 13: Esempio di albero ricorsivo per l'analisi della complessità.

La profondità parte da $d = 2 \cdot \lfloor \log_2 n \rfloor \approx 2 \cdot \log_2 n$, ovvero il suo valore massimo, proseguendo con i calcoli:

$$\begin{aligned} T(n, d) &= \sum_{i=0}^{d-1} b(n-i) = b \cdot \left(d \cdot n - \frac{d(d-1)}{2} \right) \\ T(n, d) &= b \cdot n \cdot 2 \log_2 n - b \cdot \frac{(2 \log_2 n)(2 \log_2 n - 1)}{2} \end{aligned}$$

Consideriamo la somma dell'equazione finale con il caso base di *HeapSort*, ovvero $\Theta(n \log n)$:

$$\begin{aligned} T(n, d) &= \Theta(n \log n) - \Theta((\log n)^2) + \Theta(n \log n) \\ &= \Theta(n \log n) \end{aligned}$$

Concludiamo che:

$$\text{INTROSORT_REC}(A, 1, n, \text{max_depth}) = \Theta(n \log n)$$

Considerando infine la procedura **INTROSORT**, che effettua solo il calcolo della profondità massima e una singola chiamata a **INTROSORT_REC**, la complessità complessiva diventa:

$$\begin{aligned} T(n) &= \overbrace{\Theta(1)}^{\text{calcolo della max_depth}} + \overbrace{\Theta(n \log n)}^{\text{chiamata a INTROSORT_REC}} \\ &= \Theta(n \log n) \end{aligned}$$

Analogamente, è possibile calcolare anche il costo del caso medio e migliore, i quali risultano essere sempre $\Theta(n \log n)$.

6.3 Grafici in funzione di n

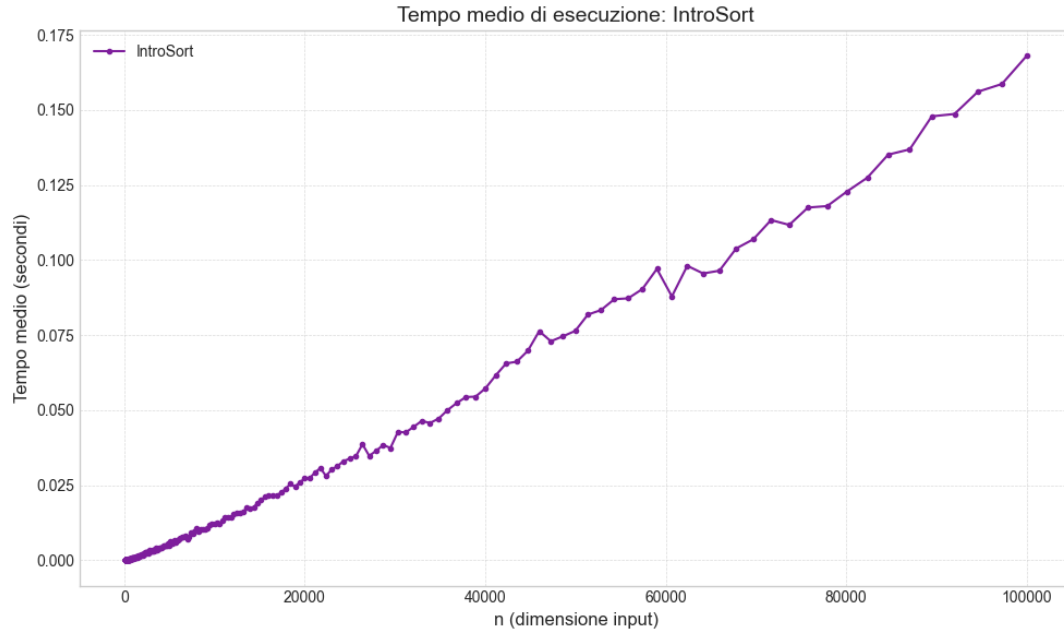


Figura 14: Scala lineare

- L'andamento è una curva crescente, che sembra essere molto vicina a una linea retta. Questo suggerisce che *IntroSort*, nel range di n considerato, si comporta come se avesse una complessità quasi lineare. La crescita del tempo di esecuzione è molto regolare, con una pendenza quasi costante.

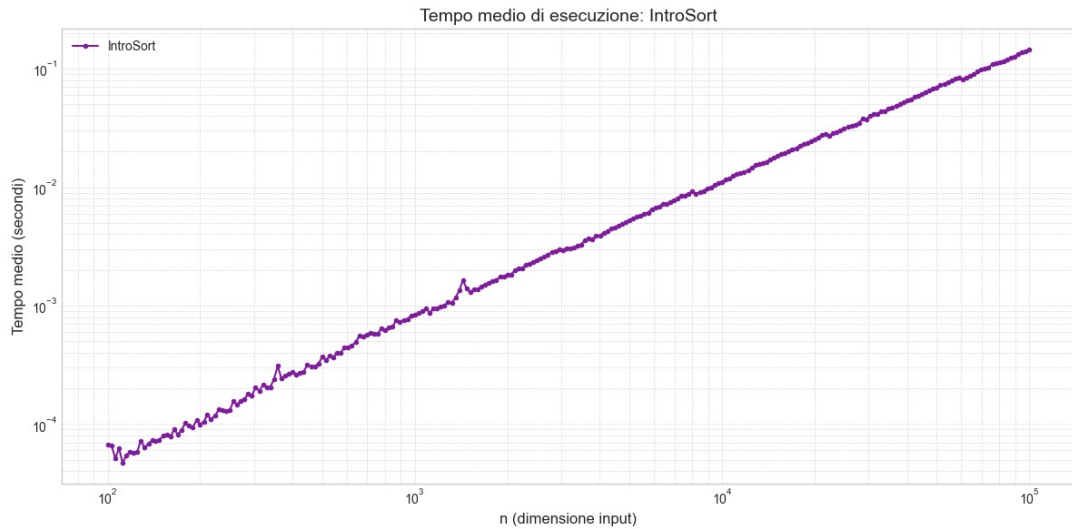


Figura 15: Scala doppiamente logaritmica

- Anche in questo caso, l'andamento è una linea retta con una pendenza costante, il che indica che l'algoritmo si comporta in modo asintoticamente stabile su tutto il range di dati. La pendenza di questa retta non è 1 (come sarebbe per una complessità lineare), ma è leggermente superiore.

6.4 Grafici in funzione di m

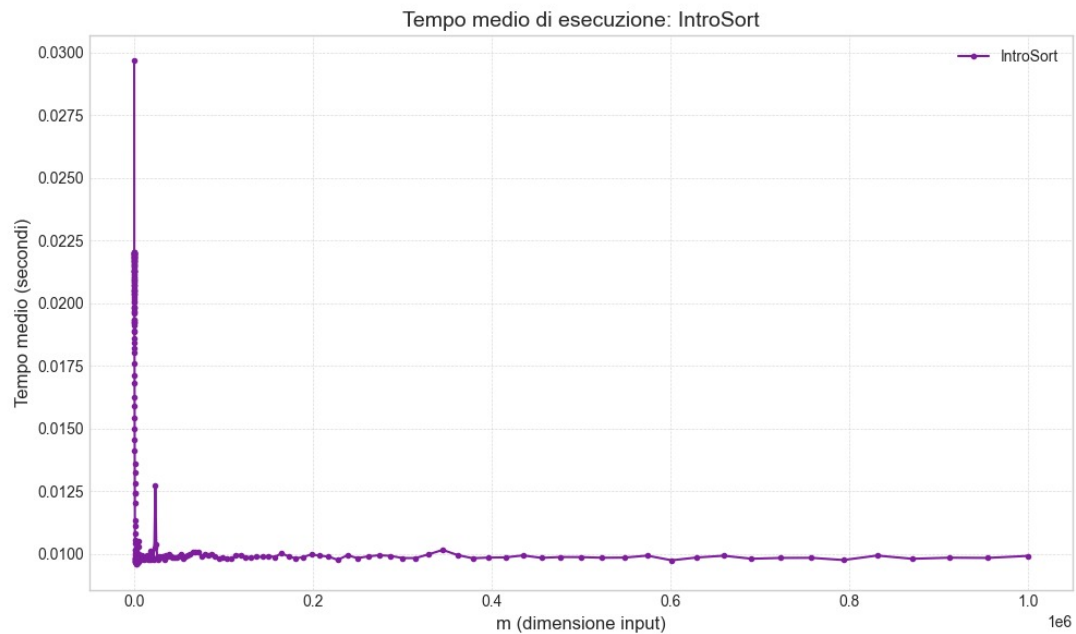


Figura 16: Scala lineare

- Con valori di m grandi, si ha che per la maggior parte del grafico, il tempo di esecuzione è molto basso e rimane praticamente costante all'aumentare di m . Questo è il comportamento atteso, poiché la complessità di *IntroSort* (come quella di *Quick-*

sort) dipende dal numero di elementi da ordinare (n , che in questo caso è fisso), non dalla dimensione dei loro valori.

- Per m piccoli, invece, si osserva un'anomalia: in questo intervallo, il tempo di esecuzione è significativamente più alto, con un picco molto evidente. Questo accade perché con un range di valori limitato, ci sono molti dati duplicati. La presenza di un'elevata quantità di elementi uguali rende il partizionamento dell'algoritmo meno efficiente, causando un aumento del tempo di esecuzione. Quando m aumenta, la probabilità di duplicati diminuisce e il tempo si stabilizza rapidamente.

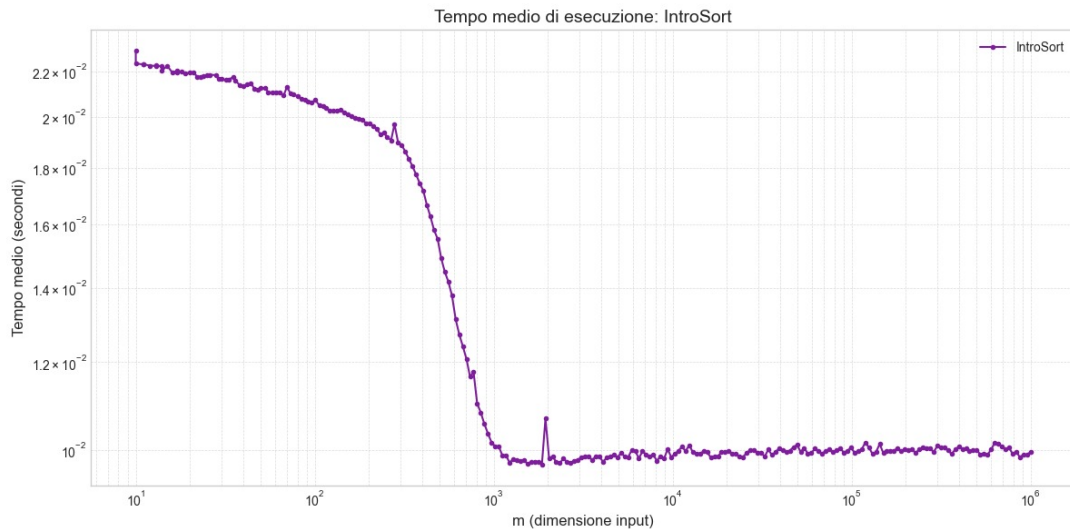


Figura 17: Scala doppiamente logaritmica

- Per valori di m molto grandi, il tempo di esecuzione è alquanto basso e rimane costante.
- C'è una rapida diminuzione del tempo di esecuzione in un intervallo ristretto: è il punto in cui l'algoritmo passa da una fase in cui gestisce molti elementi duplicati (che lo rallentano) a una fase in cui la distribuzione dei valori è sufficientemente dispersa da permettergli di operare in modo efficiente.
- Per m piccoli, il tempo di esecuzione è più alto e diminuisce in modo graduale all'aumentare di m . Questo comportamento è causato dal gran numero di duplicati che rallentano l'algoritmo, come già discusso per il grafico lineare.

7 Confronto tra algoritmi

7.1 *QuickSort* VS *QuickSort 3-Way*

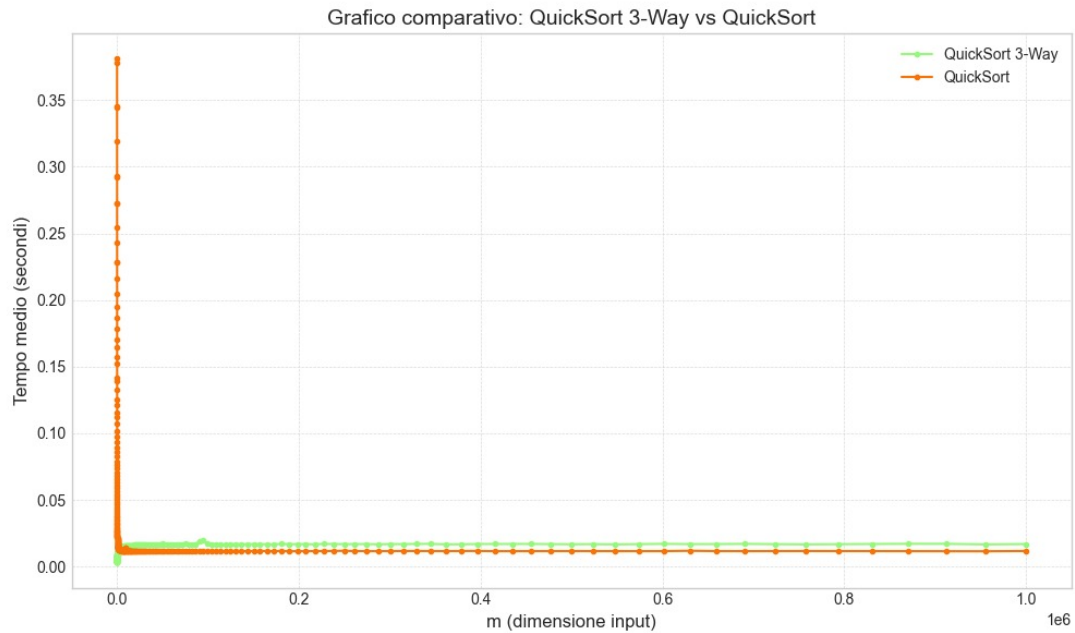


Figura 18: Scala lineare, grafico in funzione di m

- Questo grafico mette a confronto gli algoritmi *QuickSort* e *QuickSort 3-Way*, mostrando chiaramente un'ottima stabilità ed efficienza nel caso di un partizionamento a 3 vie, battendo il *QuickSort* tradizionale, il quale presenta all'inizio un netto rallentamento, come analizzato in precedenza.

7.2 *IntroSort* VS *CountingSort*

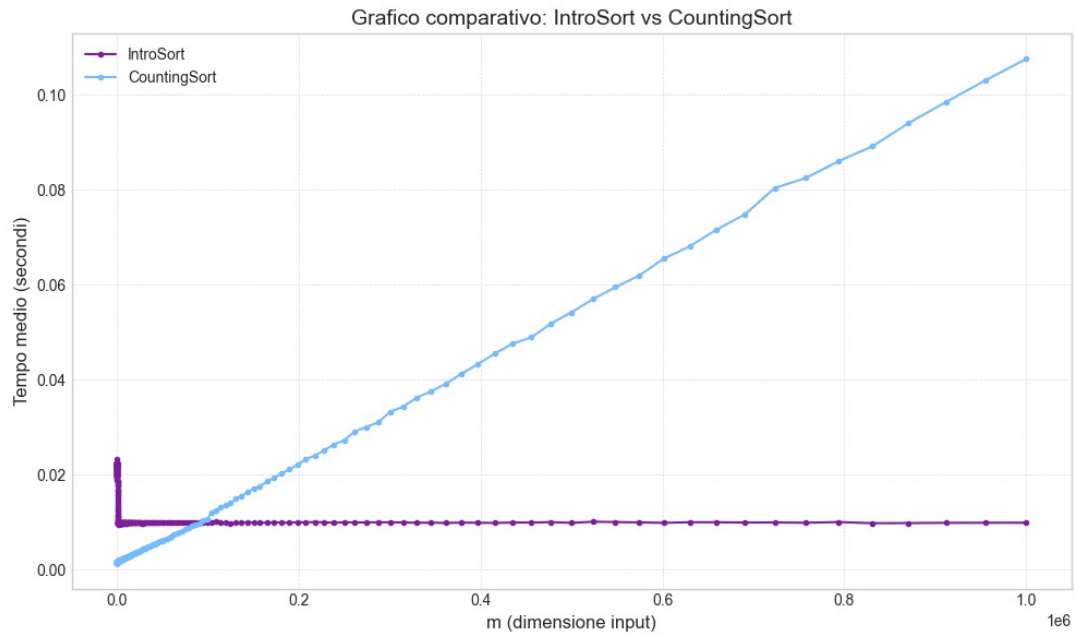


Figura 19: Scala lineare, grafico in funzione di m

- Il grafico compara *IntroSort* e *CountingSort* in funzione di m : il primo mostra un tempo di esecuzione che si stabilizza rapidamente su un valore basso e costante. Questo perché la sua complessità non dipende da m .
Il secondo mostra un tempo di esecuzione che cresce in modo lineare. Questo per via della sua complessità, e siccome m è il parametro variabile e n è fisso, il tempo di esecuzione cresce linearmente con m .
- *IntroSort* è molto più efficiente in questo scenario, perché il suo tempo di esecuzione è indipendente da m . *CountingSort*, al contrario, diventa sempre più lento all'aumentare del range dei valori.

7.3 QuickSort VS CountingSort

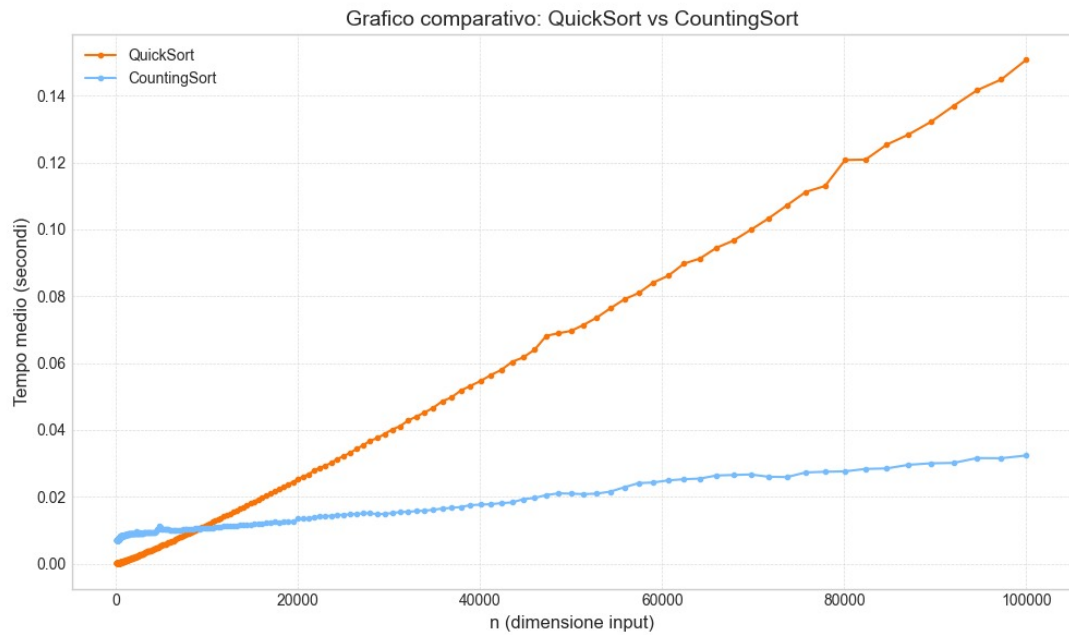


Figura 20: Scala lineare, grafico in funzione di n

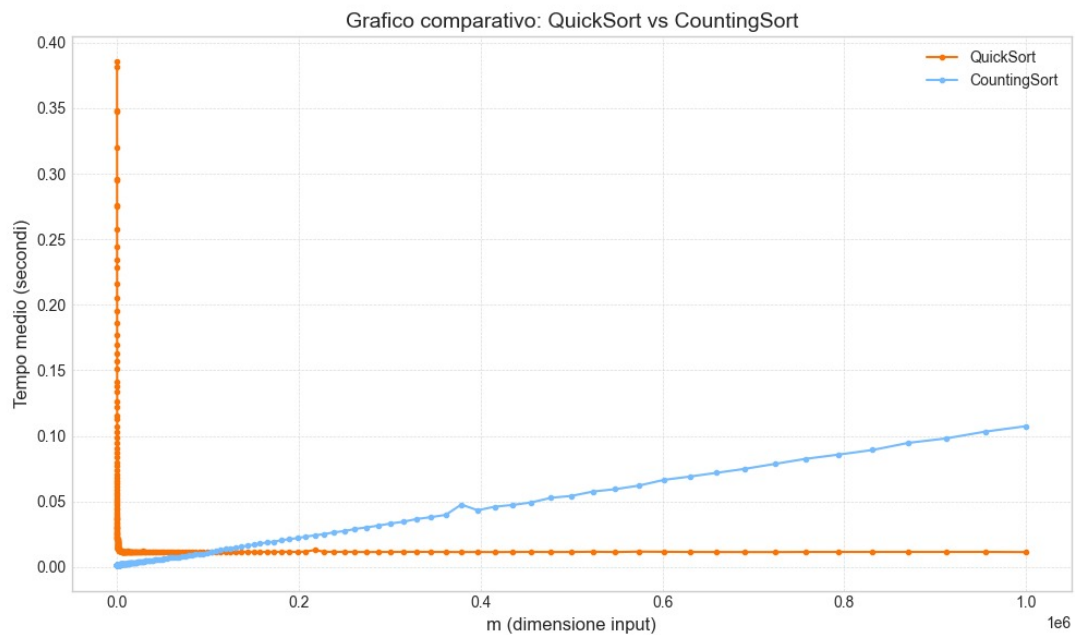


Figura 21: Scala lineare, grafico in funzione di m

- In questo confronto vengono comparati due algoritmi asintoticamente differenti: *QuickSort*, avente una complessità $\Theta(n \log n)$, e *CountingSort* con una complessità $\Theta(n + k)$. Effettivamente, nel primo grafico possiamo evincere che *CountingSort* è significativamente più veloce di *QuickSort*, infatti presenta una linea quasi crescente linearmente (per $n = 100.000$ impiega circa 0.03s, contro *QuickSort* con 0.15s); soprattutto nel caso di m molto piccolo.

- D'altra parte, nel secondo grafico possiamo affermare che per un m molto grande *QuickSort* diventa competitivo e più efficiente, probabilmente perché l'*overhead* di *CountingSort*, legato alla dimensione m , diventa più significativo rispetto al numero di elementi da ordinare.

7.4 Tutti

7.4.1 Grafici in funzione di n

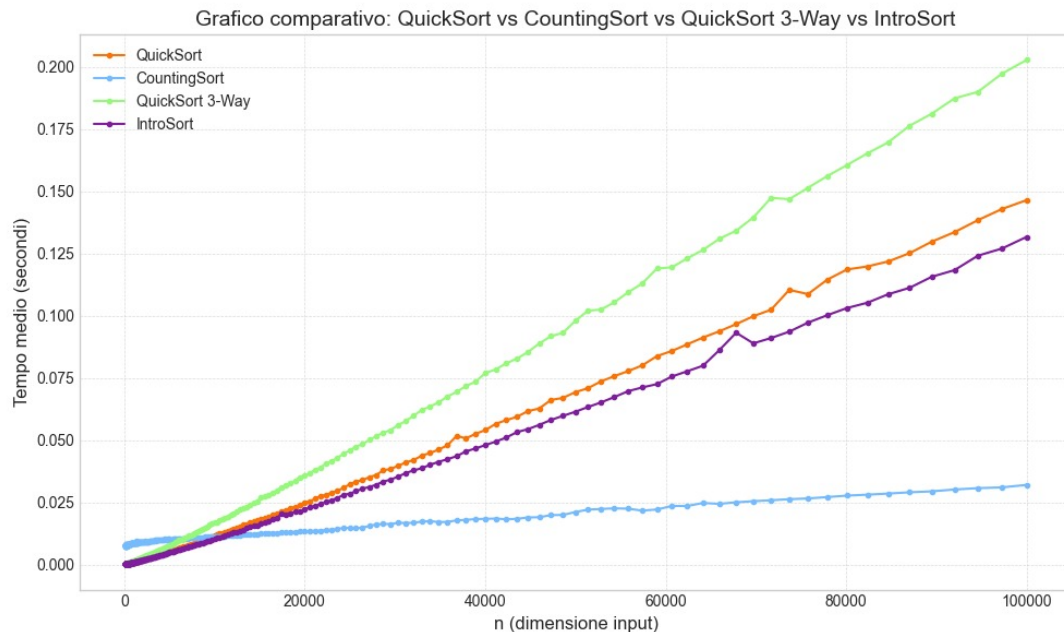


Figura 22: Scala lineare

- Il grafico confronta il tempo di esecuzione di quattro algoritmi in funzione della dimensione dell'input n .
CountingSort è l'algoritmo più veloce e in questo caso, poiché n è il parametro variabile e m è costante, il suo tempo di esecuzione cresce quasi linearmente con n .
IntroSort e *QuickSort* mostrano un andamento simile. La loro complessità è $\Theta(n \log n)$ e, come visto in precedenza, nel range considerato sembra quasi lineare. *IntroSort* risulta leggermente più veloce di *QuickSort*.
QuickSort 3-Way è il più lento tra tutti. Rispetto alla sua complessità teorica, le costanti di tempo e le operazioni aggiuntive che esegue lo rendono meno efficiente rispetto a *QuickSort* e *IntroSort*.
- In sintesi: *CountingSort* è il più performante in questo scenario. Tra gli algoritmi basati su confronti, *IntroSort* è il più efficiente, seguito da *QuickSort*, mentre *QuickSort 3-Way* si dimostra il più lento.

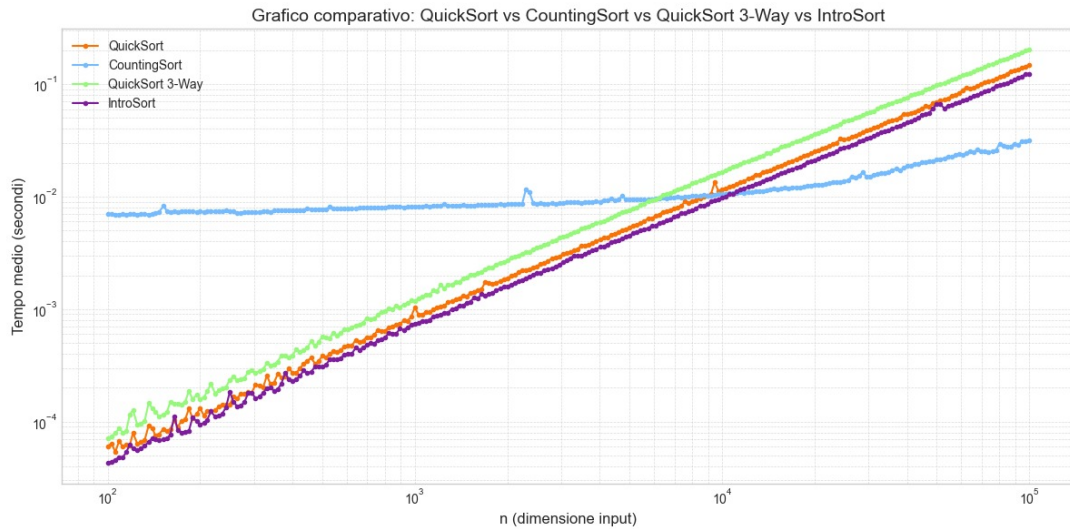


Figura 23: Scala doppiamente logaritmica

- Il grafico evidenzia che per la tipologia di dati utilizzata in questo test, *IntroSort* e *QuickSort* sono le scelte più performanti. Nonostante la sua complessità teorica favorevole in certi contesti, il *CountingSort* si dimostra l'algoritmo meno efficiente in questo scenario specifico.
- *QuickSort 3-Way* mostra prestazioni leggermente inferiori rispetto a *QuickSort* e *IntroSort* in questo specifico test. Sebbene sia teoricamente vantaggioso per dati con pochi valori unici, potrebbe avere un *overhead* maggiore che lo rende meno performante in scenari più generali.

CountingSort invece mostra un comportamento nettamente diverso: la sua linea è quasi piatta, indicando che il tempo di esecuzione è meno sensibile all'aumento della dimensione dell'input n , il che è coerente con la sua complessità temporale lineare. Tuttavia, in questo *benchmark*, risulta costantemente più lento degli altri algoritmi, ad eccezione delle dimensioni di input molto piccole. Questo suggerisce che il costo fisso o la dipendenza dall'intervallo m lo rendono meno competitivo rispetto agli algoritmi basati su confronti per i dati testati.

7.4.2 Grafici in funzione di m

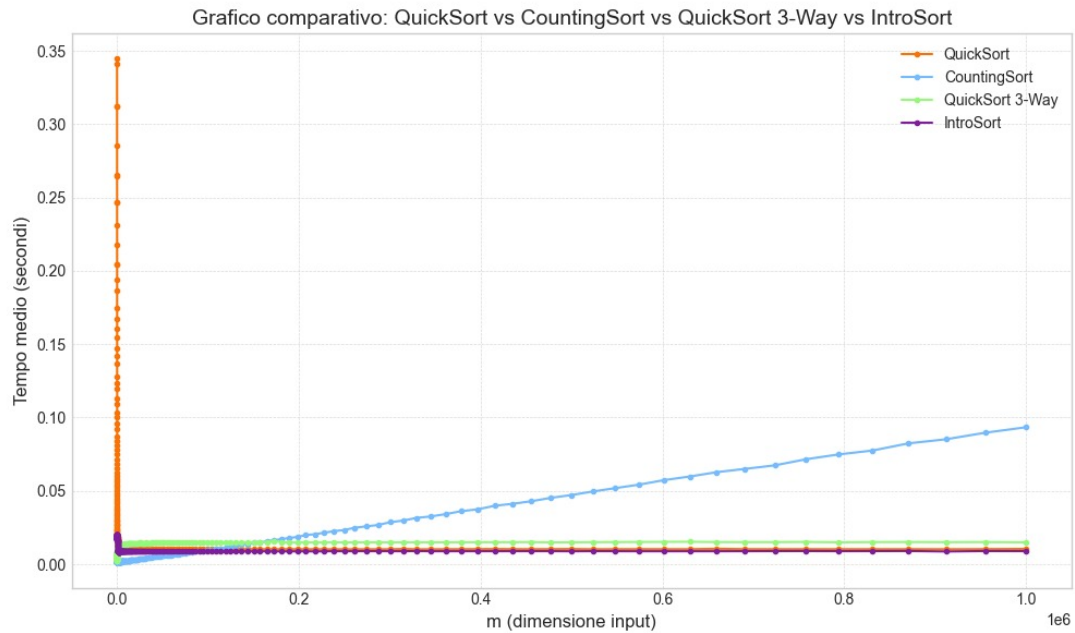


Figura 24: Scala lineare

- Questo grafico mostra in modo evidente che il tempo di esecuzione di *CountingSort* aumenta linearmente con l'aumentare del range m . Le prestazioni degli altri algoritmi, invece, vengono appiattite a ridosso dell'asse delle ascisse, diventando difficili da distinguere, ad eccezione di un picco anomalo di *QuickSort* per valori molto piccoli di m .

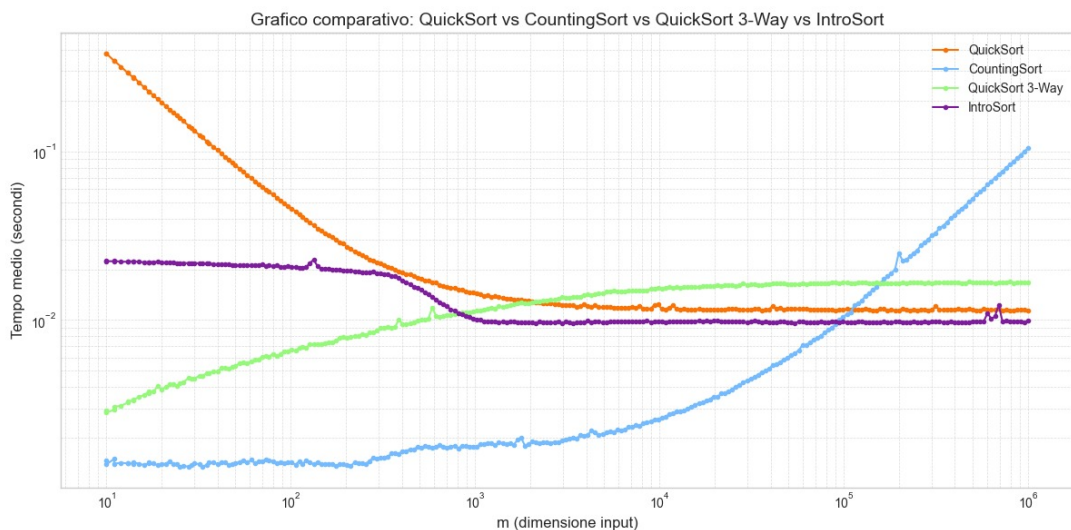


Figura 25: Scala doppiamente logaritmica

- *CountingSort* conferma il suo andamento: è il più veloce per range di valori molto piccoli, ma le sue prestazioni degradano rapidamente all'aumentare di m .

QuickSort e *IntroSort* si comportano in modo opposto: sono lenti quando il range m è piccolo (il che implica un'alta presenza di elementi duplicati), ma diventano progressivamente più veloci ed efficienti all'aumentare del range, superando tutti gli altri.

QuickSort 3-Way dimostra la sua efficacia nel gestire i duplicati, infatti risulta essere più veloce di *QuickSort* ed *IntroSort* per range piccoli e medi, ma viene superato quando i valori diventano più diversificati (range m grande).